Позиционирование элементов

CSS рассматривает макет html-документа как дерево элементов. Уникальный элемент, у которого нет родительского элемента, называется **корневым** элементом. Модуль CSS-позиционирование описывает, как любой из элементов может быть размещен независимо от порядка документа (т.е. извлечен из «потока»).

- Обычно элементы идут друг за другом в том порядке, в котором они написаны в HTML. Это называется нормальным потоком.
- Позиционирование позволяет изменить это поведение.
- Мы можем:
  - Сдвинуть элемент относительно его обычного места.
  - Изъять элемент из потока, чтобы он не влиял на соседей.
  - Закрепить элемент в окне браузера, чтобы он всегда был виден.
  - Размещать элементы друг на друге.

Схемы позиционирования

1. Нормальный поток
2. Обтекание
3. Абсолютное позиционирование

Нормальный поток

Нормальный поток включает блочный контекст форматирования (элементы с `display` `block`, `list-item` или `table`), строчный (встроенный) контекст форматирования (элементы с `display` `inline`, `inline-block` или `inline-table`), и относительное и «липкое» позиционирование элементов уровня блока и строки.

Обтекание

В обтекающей модели блок удаляется из нормального потока и позиционируется влево или вправо. Содержимое обтекает правую сторону элемента с `float: left` и левую сторону элемента с `float: right`.

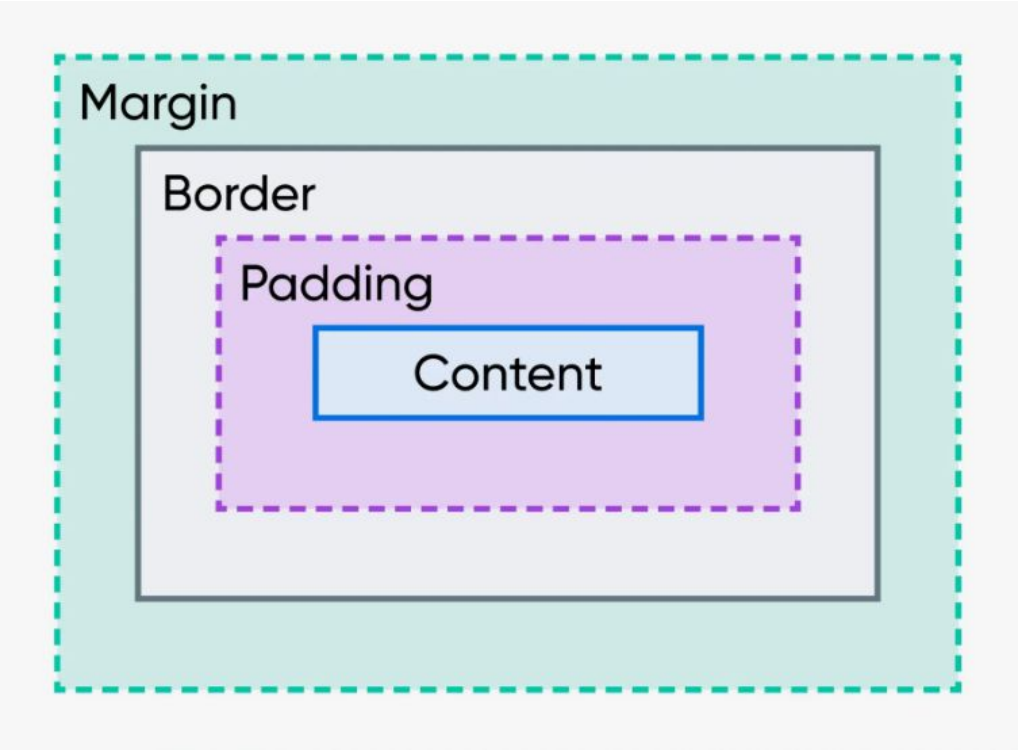
Абсолютное позиционирование

В модели абсолютного позиционирования блок полностью удаляется из нормального потока и ему присваивается позиция относительно содержащего блока. Абсолютное позиционирование реализуется с помощью значений `position: absolute;` и `position: fixed;`.

Элементом «вне потока» может быть плавающий, абсолютно позиционированный или корневой элемент.

Содержащий блок

Положение и размер блока(ов) элемента иногда вычисляются относительно некоторого прямоугольника, называемого **содержащим блоком** элемента (containing block). Содержащий блок — это блок, который содержит другой элемент. В случае нормального потока корневой элемент html является содержащим блоком для элемента body, который, в свою очередь, является содержащим блоком для всех его дочерних элементов и так далее. В случае позиционирования содержащий блок полностью зависит от типа позиционирования



Содержащий блок

- Содержащий блок, в котором находится корневой элемент, представляет собой прямоугольник — так называемый **начальный содержащий блок**.
- Для некорневого элемента с `position: static;` или `position: relative;` содержащий блок формируется краем области содержимого ближайшего родительского блока уровня блока, ячейки таблицы или уровня строки.
- Содержащим блоком элемента с `position: fixed;` является окно просмотра.
- Для некорневого элемента с `position: absolute;` содержащим блоком устанавливается ближайший родительский элемент со значением `position: absolute/relative/fixed` следующим образом:
  — если предок – элемент уровня блока, содержащим блоком будет область содержимого плюс поля элемента `padding`;
  — если предок – элемент уровня строки, содержащим блоком будет область содержимого;
  — если предков нет, то содержащий блок элемента определяется как начальный содержащий блок.
- Для «липкого» блока содержащим блоком является ближайший предок с прокруткой или окно просмотра, в противном случае

Типы позиционирования: свойство position

Для активации специфического типа позиционирования у элемента, мы используем свойство `position`. Свойство `position` определяет, какой из алгоритмов позиционирования используется для вычисления положения блока.

Свойство не наследуется.

Статическое позиционирование

static Блок располагается в соответствии с нормальным потоком. Свойства top, right, bottom и left не применяются. Значение по умолчанию.

Пример

HTML: `<p class="positioned">...</p>`

CSS:

```
.positioned {
    position: static;
    background: red;
}
```

Никаких изменений не произойдет, так как статическое позиционирование является поведением по умолчанию

static (по умолчанию)

```
position: static;
```

Относительное позиционирование

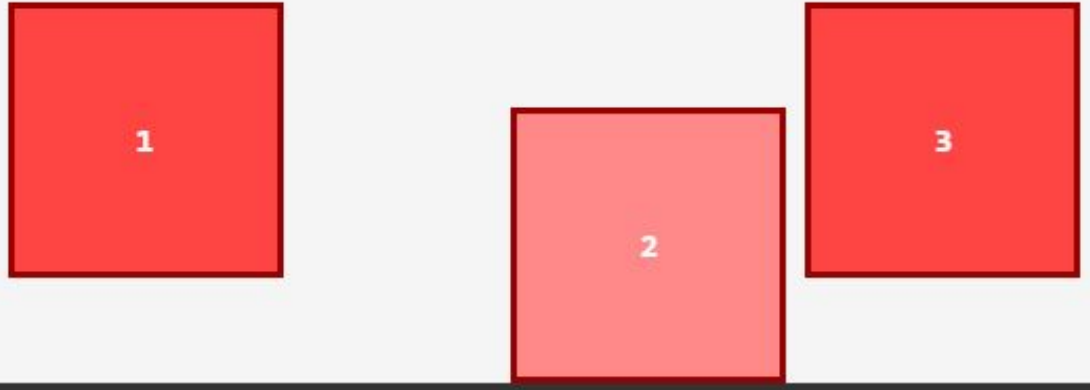
relative Положение блока рассчитывается в соответствии с нормальным потоком. Затем блок смещается относительно его нормального положения и во всех случаях, включая элементы таблицы, не влияет на положение любых следующих блоков. Тем не менее, такое смещение может привести к перекрытию блоков, а также к появлению полосы прокрутки в случае переполнения.

- Сохраняет свои размеры, включая разрывы строк и пространство, первоначально зарезервированное для него.
- Создает новый содержащий блок для абсолютно позиционированных потомков.

Влияние position: relative; на элементы таблицы определяется следующим образом:

- Элементы с table-row-group, table-header-group, table-footer-group ид table-row смещаются относительно их обычной позиции в таблице.
- Если ячейки таблицы занимают несколько строк, смещаются только ячейки начальной строки.
- table-column-group, table-column не смещает соответствующий столбец и не оказывает визуального влияния.
- table-caption and table-cell смещаются относительно своего нормального положения в таблице. Если ячейка таблицы охватывает несколько столбцов или строк, то она смещается целиком.

relative




```
position: relative;
top: 40px;
left: 40px;
```

top:40

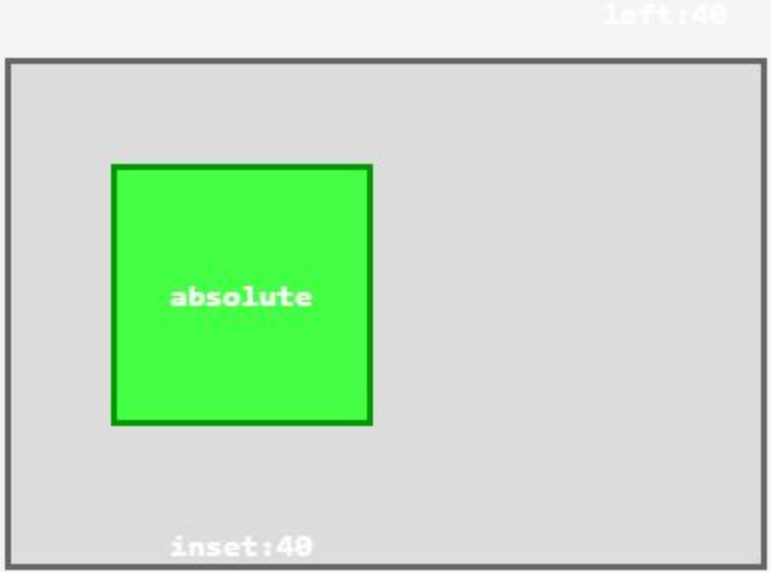
Абсолютное позиционирование

absolute Положение блока (и, возможно, размер) задается с помощью свойств top, right, bottom и left. Эти свойства определяют явное смещение относительно его содержащего блока. Абсолютно позиционированные блоки полностью удаляется из нормального потока, не влияя на расположение сестринских элементов.
Отступы margin абсолютно позиционированных блоков не схлопываются.

Абсолютно позиционированный блок создает новый содержащий блок для дочерних элементов нормального потока и потомков с position: absolute;.

Содержимое абсолютно позиционированного элемента не может обтекать другие блоки. Абсолютно позиционированный блок могут скрывать содержимое другого блока (или сами могут быть скрыты), в зависимости от значения z-index перекрывающихся блоков.

absolute




```
.container { position: relative; }
.square {
  position: absolute;
  inset-inline-start: 40px;
  inset-block-start: 40px;
}
```

absolute внутри relative

- Если просто сделать дочерний элемент absolute, он будет позиционироваться относительно окна браузера (<html>), что не всегда удобно. Чтобы "привязать" абсолютный элемент к родителю, нужно сделать этого родителя позиционированным, чаще всего — position: relative; без смещен

Контекст позиционирования — относительно какого элемента позиционируется позиционируемый элемент. Это делается путём установки позиционирования на одном из предков элемента — на один из элементов, внутри которого он вложен (вы не можете позиционировать его относительно элемента, внутри которого он НЕ вложен).

absolute внутри relative

```
.parent {
    position: relative; /* Родитель становится точкой отсчета */
    border: 2px solid blue;
    height: 200px;
}

.child {
    position: absolute;
    bottom: 10px; /* Ребенок прижмется к низу родителя */
    right: 10px;  /* и к правому краю */
    background: red;
}
```

Это частый паттерн использования абсолютного позиционирования. Мы делаем родительский элемент position: relative; (при этом он остается в потоке), а дочерний — position: absolute;. Теперь координаты дочернего элемента будут отсчитываться от границ родителя, а не от всего окна браузера. Это позволяет, например, надежно прикрепить иконку или подсказку к определенному углу карточки.

absolute внутри relative

```
.parent {
  position: relative; /* Родитель становится точкой отсчета */
  border: 4px solid blue;
  height: 250px;
}

.child {
  position: absolute;
  bottom: 20px; /* Ребенок прижимается к низу родителя */
  right: 20px; /* и к правому краю */
  background: red;
}
```

Фиксированное позиционирование

fixed Фиксированное позиционирование аналогично абсолютному позиционированию, с отличием в том, что для содержащим блоком устанавливается окно просмотра. Такой блок полностью удаляется из потока документа и не имеет позиции относительно какой-либо части документа. Фиксированные блоки не перемещаются при прокрутке документа. В этом отношении они похожи на фиксированные фоновые изображения.

При печати фиксированные блоки повторяются на каждой странице, содержащим блоком для них устанавливается область страницы. Блоки с фиксированным положением, которые больше области страницы, обрезаются.

fixed (в углу экрана)

При прокрутке страницы - розовый квадрат в углу не двигается

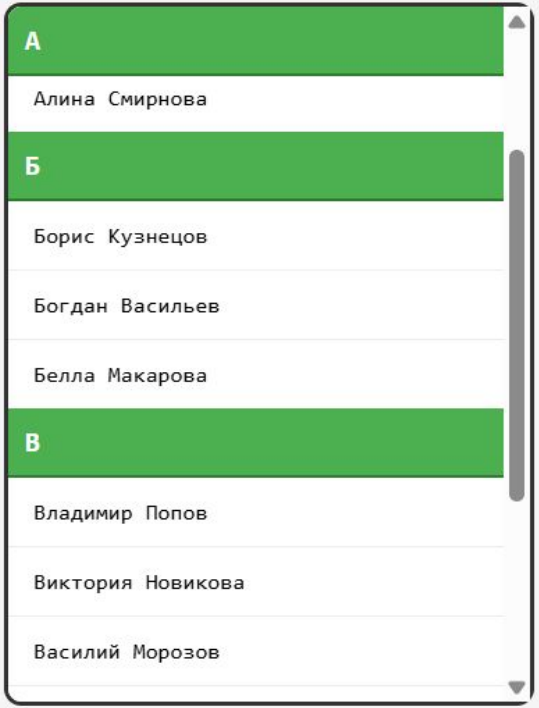
```
position: fixed;
bottom: 20px;
right: 20px;
```

fixed

«Приклеенное» позиционирование

sticky Положение блока рассчитывается в соответствии с нормальным потоком. Затем блок смещается относительно своего ближайшего предка с прокруткой или окна просмотра, если ни у одного из предков нет прокрутки.
«Липкий» блок может перекрывать другие блоки, а также создавать полосы прокрутки в случае переполнения.
«Липкий» блок сохраняет свои размеры, включая разрывы строк и пространство, первоначально зарезервированное для него.
«Липкий» блок создает новый содержащий блок для абсолютно и относительно позиционированных потомков.

sticky (как в списке контактов)




```
/* Заголовки прилипают при прокрутке */
.letter-header {
  position: sticky;
  top: 0;
  background: #4CAF50;
}
```

Координаты

Элемент считается позиционированным, если свойство `position` имеет значение, отличное от `static`. Позиционированные элементы генерируют позиционированные блоки и могут быть расположены в соответствии со следующими четырьмя физическими свойствами:

1. **top**
2. **right**
3. **bottom**
4. **left**

top

Свойство top задает расстояние, на которое верхний край абсолютно позиционированного блока (с учетом его margin) смещается ниже верхнего края содержащего блока. Для относительно позиционированных блоков определяет смещение относительно верхнего края самого блока (то есть блоку задается позиция в нормальном потоке, а затем смещение от этой позиции в соответствии с этим свойством):

- **auto** Влияние значения зависит от типа элемента. Значение по умолчанию.
- **длина** Смещение на фиксированном расстоянии от указанного края. Отрицательные значения допускаются.
- **%** Процентные значения вычисляются относительно высоты содержащего блока. Для «липкого» блока — относительно высоты корневого элемента. Отрицательные значения допускаются.
- **initial** Устанавливает значение свойства в значение по умолчанию.

right

Свойство right указывает расстояние, на которое правый край абсолютно позиционированного блока (с учетом его margin) смещен влево от правого края содержащего блока. Для относительно позиционированных блоков определяет смещение относительно правого края самого блока.

- **auto** Влияние значения зависит от типа элемента. Значение по умолчанию.
- **длина** Смещение на фиксированном расстоянии от указанного края. Отрицательные значения допускаются.
- **%** Процентные значения вычисляются относительно ширины содержащего блока. Для «липкого» блока — относительно ширины корневого элемента. Отрицательные значения допускаются.
- **initial** Устанавливает значение свойства в значение по умолчанию

bottom

Свойство bottom указывает расстояние, на которое нижний край блока смещен вверх относительно нижнего края содержащего блока. Для относительно позиционированных блоков определяет смещение относительно нижнего края самого блока.

- **auto** Влияние значения зависит от типа элемента. Значение по умолчанию.
- **длина** Смещение на фиксированном расстоянии от указанного края. Отрицательные значения допускаются.
- **%** Процентные значения вычисляются относительно высоты содержащего блока. Для «липкого» блока — относительно высоты корневого элемента. Отрицательные значения допускаются.
- **initial** Устанавливает значение свойства в значение по умолчанию.

left

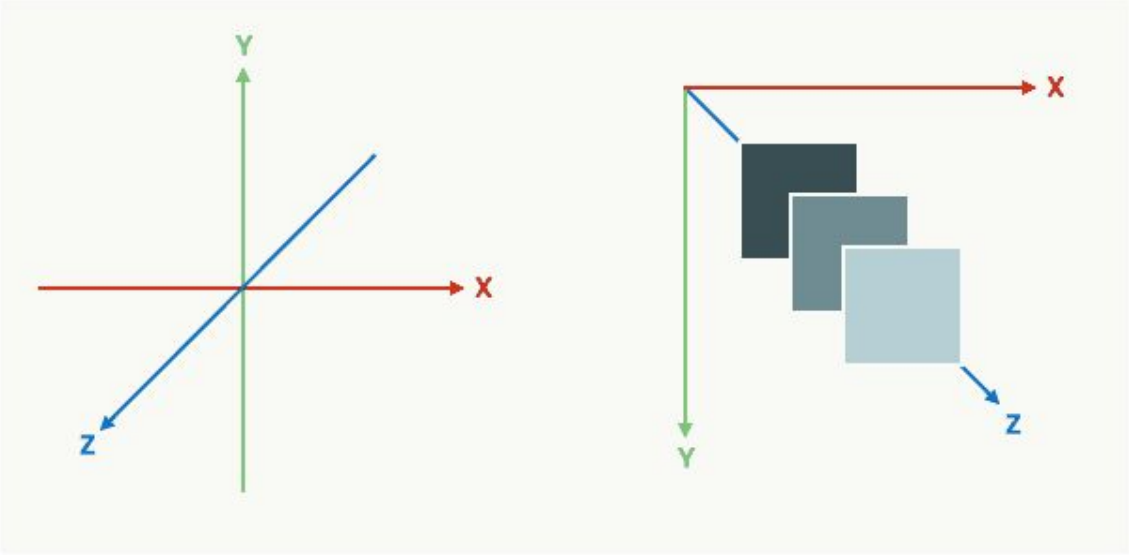
Свойство `left` указывает расстояние, на которое левый край смещен вправо от левого края содержащего блока. Для относительно позиционированных блоков определяет смещение относительно левого края самого блока.

Положительные значения смещают элемент внутрь содержащего блока, а отрицательные — за его пределы.

- **auto** Влияние значения зависит от типа элемента. Значение по умолчанию.
- **длина** Смещение на фиксированном расстоянии от указанного края. Отрицательные значения допускаются.
- **%** Процентные значения вычисляются относительно ширины содержащего блока. Для «липкого» блока — относительно ширины корневого элемента. Отрицательные значения допускаются.
- **initial** Устанавливает значение свойства в значение по умолчанию.

Определение контекста наложения: свойство z-index

В CSS каждый блок имеет позицию в трех измерениях. В дополнение к горизонтальному и вертикальному положению, блоки выкладываются вдоль оси Z друг над другом. Положение вдоль оси Z особенно важно, когда блоки визуально накладываются друг на друга.



Определение контекста наложения: свойство z-index

Порядок, в котором дерево документа отрисовывается на экране, описывается с помощью **контекста наложения**. Каждый блок принадлежит одному контексту наложения. Каждый блок в данном контексте наложения имеет целочисленный уровень, который является его положением на оси Z относительно других блоков в том же контексте наложения.

Блоки с более высокими уровнями всегда отображаются перед блоками с более низкими уровнями, а блоки с одинаковым уровнем располагаются снизу вверх в соответствии с порядком следования элементов в исходном документе. Блок элемента имеет ту же позицию, что и блок его родителя, если только ему не присвоен другой уровень свойством `z-index`.

Определение контекста наложения: свойство z-index

Свойство `z-index` позволяет изменить порядок наложения позиционированных элементов в случае, когда они накладываются друг на друга.

Свойство не наследуется.

- **auto** Вычисляется в 0. Если для блока задано position: fixed; или это корневой элемент, значение auto также устанавливает новый контекст наложения. Значение по умолчанию.
- **целое число** Определяет положение блока в текущем контексте наложения. Также устанавливает новый локальный контекст наложения. Можно использовать любое целое число, включая отрицательные числа. Отрицательные значения помещают элемент вглубь экрана.
- **initial** Устанавливает значение свойства в значение по умолчанию.

Контекст наложения

Если для элементов свойства z-index и position не заданы явно, контекст наложения равен порядку их расположения в исходном коде и браузер отображает элементы на странице в следующем порядке:

- Корневой элемент <html>, который содержит все элементы веб-странице.
- Блочные элементы, неплавающие и непозиционированные.
- Плавающие float непозиционированные элементы в порядке их расположения в исходном коде.
- Строковые непозиционированные элементы (текст, изображения).
- Позиционированные position элементы в порядке их следования в исходном коде. Последний из них будет расположен на переднем плане.

Контекст наложения

Свойство z-index создает новый контекст наложения. Оно позволяет изменить порядок наложения позиционированных элементов. Элементы будут отображаться на странице в следующем порядке (если для них не заданы свойства, влияющие на контекст наложения — opacity, filter, transform):

- Корневой элемент <html>, который содержит все элементы веб-странице.
- Позиционированные элементы с отрицательным значением z-index.
- Блочные элементы, неплавающие и непозиционированные.
- Плавающие float непозиционированные элементы в порядке их расположения в исходном коде.
- Строковые непозиционированные элементы (текст, изображения).
- Позиционированные элементы со значениями z-index: 0; и z-index: auto;

Таблицы

Границы таблицы border

Таблица и ячейки внутри неё по умолчанию отображаются в браузере без видимых границ — они есть, но их не видно. Чтобы их увидеть, мы используем привычное свойство `border`, которое задает рамку: ее толщину, стиль и цвет.

```
border: 2px dashed blue;    /* толщина | стиль | цвет */
```

Как задать ширину и высоту таблицы

По умолчанию **ширина и высота таблицы** определяется содержимым её ячеек. Если ширина не задана, то она будет равна ширине самого широкого ряда (строки).

Ширина таблицы и столбцов задаётся с помощью свойства width. Если для таблицы задано table {width: 100%;}, то ширина таблицы будет равна ширине блока-контейнера, в котором она находится.
Ширину таблицы и столбцов обычно задают в px или %, например:

```
table {
    width: 600px;
}
```

Высота таблицы не задается. **Высотой рядов** таблицы можно управлять, добавив верхний и нижний padding для элементов <td> и <th>. Фиксировать высоту с помощью свойства height не рекомендуется.

```
th, td {
    padding: 10px 15px;
}
```

Столбцы таблицы

Модель CSS таблиц ориентирована в основном на строки (ряды), формируемые с помощью элемента <tr>. На практике бывают случаи, когда необходимо специальное форматирование столбцов, которое возможно следующими способами:

- с помощью элемента <col> можно задать фон для любого количества столбцов;
- с помощью селектора table td:first-child, table td:last-child можно задать стили для первого или последнего столбца таблицы (за исключением первой ячейки заголовка таблицы);
- с помощью селектора table td:nth-child(правило отбора столбцов) можно задать стили для любых столбцов таблицы.

Как убрать промежуток между рамками ячеек: border-collapse

Рамки ячеек таблицы по умолчанию разделены небольшим промежутком. Если задать для таблицы `border-collapse: collapse`, то промежуток уберётся.

Свойство наследуется.

- **separate** Рамки ячеек располагаются раздельно.
- **collapse** Рамки ячеек сливаются в одну, а промежутки между рамками убираются.
- **initial** Устанавливает значение свойства в значение по умолчанию.

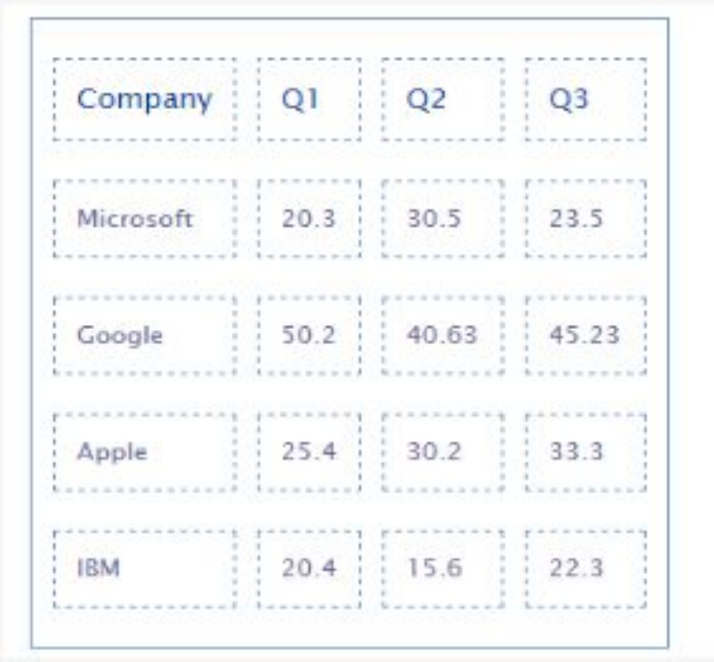
Company	Q1	Q2	Q3
Microsoft	20.3	30.5	23.5
Google	50.2	40.63	45.23
Apple	25.4	30.2	33.3
IBM	20.4	15.6	22.3

Как увеличить промежуток между рамками ячеек: border-spacing

С помощью свойства border-spacing можно менять расстояние между рамками ячеек. Данное свойство применяется к таблице в целом. Свойство наследуется.

- **<длина> <длина>** Добавляет промежутки между рамками как по вертикали, так и по горизонтали. Если заданы две длины, то первая всегда определяет горизонтальный промежуток, а вторая — вертикальный.
- **initial** Устанавливает значение свойства в значение по умолчанию.

```
table {
    border-collapse: separate;
    border-spacing: 10px 20px;
  }
```



Company	Q1	Q2	Q3
Microsoft	20.3	30.5	23.5
Google	50.2	40.63	45.23
Apple	25.4	30.2	33.3
IBM	20.4	15.6	22.3

Как поместить заголовок перед или под таблицей: caption-side

Добавить заголовок в таблицу можно с помощью элемента <caption>, а с помощью свойства caption-side его можно поместить перед таблицей или под ней. Для горизонтального выравнивания текста заголовка применяется свойство text-align.
Свойство наследуется.

- **top** Заголовок таблицы располагается над таблицей. Значение по умолчанию.
- **bottom** Располагает заголовок под таблицей
- **initial** Устанавливает значение свойства в значе по умолчанию.

Company	Q1	Q2	Q3	Q4
Microsoft	20.3	30.5	23.5	40.3
Google	50.2	40.63	45.23	39.3
Apple	25.4	30.2	33.3	36.7
IBM	20.4	15.6	22.3	29.3

Таблица № 1

Как выровнять содержимое внутри ячеек по вертикали: vertical-align

Даже если мы задали ячейкам одинаковую высоту, содержимое в них может располагаться по-разному. Свойство `vertical-align` управляет вертикальным выравниванием содержимого внутри ячейки таблицы.

- **top** Выравнивает содержимое по верхнему краю ячейки.
- **middle** Выравнивает содержимое по центру ячейки по вертикали. Значение по умолчанию.
- **bottom** Выравнивает содержимое по нижнему краю ячейки.
- **baseline** Выравнивает содержимое по базовой линии первой строки текста (используется редко, в основном для сложных таблиц с разным размером шрифта).
- **initial** Устанавливает значение свойства в значение по умолчанию.

Как скрыть пустые ячейки таблицы

Свойство empty-cells скрывает или показывает пустые ячейки. Действует только на ячейки, которые не содержат какой-либо контент. Если для ячейки задан фон, а для таблицы задано table {border-collapse: collapse;}, то ячейка не будет скрыта. Свойство наследуется.

- **show** Рамка и фон пустой ячейки будут отрисовываться так же, как для ячейки таблицы, имеющей содержимое.
- **hide** Если все ячейки строки пусты, то вся строка отображается так, если бы было задано значение display: none.
- **initial** Устанавливает значение свойства в значение по умолчанию.

Company	Q1	Q2	Q3
Microsoft	20.3	30.5	
Google	50.2	40.63	45.23

Компоновка макета таблицы с помощью свойства table-layout

Компоновка макета таблицы определяется одним из двух подходов: фиксированный макет или автоматический макет. Под компоновкой в данном случае подразумевается как распределяется ширина таблицы между шириной ячеек. Свойство не наследуется.

- **auto** Значение по умолчанию. Ширина макета таблицы определяется шириной её содержимого с учетом значений свойств padding-left, padding-right, border-left width плюс одна ширина border-right последней ячейки в ряду, или заданной шириной ячеек и толщиной рамки. Если ширина ячеек не задана явно, они могут быть разной ширины.
- **fixed** Свойство сработает только в том случае, если для таблицы задана ширина. Ширина ячеек будет одинаковой, а содержимое ячеек, которое не помещается в ячейку, будет наползать под содержимое соседней ячейки.

```
table {
    table-layout: fixed;
}
```

Центрирование элементов

В CSS не существует одного универсального способа отцентрировать элемент. Выбор метода зависит от типа элемента (блочный или строчный), от родителя и от того, по какой оси нужно центрировать.

Центрирование строчных элементов по горизонтали

Для строчных элементов (inline, inline-block). Свойство text-align: center на родительском элементе.

```
.parent {
    text-align: center;
    border: 1px solid blue;
}
<div class="parent">
    <span>Я по центру</span>
    <img src="photo.jpg" alt="Картинка">
    <button>Кнопка</button>
</div>
```

Подходит для: текста, картинок, кнопок, inline-block элементов

Центрирование блочных элементов по горизонтали

Для блочных элементов (`block`). Свойство `margin: 0 auto` на самом элементе.

Запись `margin: 0 auto` *. Первое значение (0) задает отступы сверху и снизу, второе (auto) — слева и справа.*

```
.block {
    width: 300px;        /* Обязательно задать ширину */
    margin: 0 auto;      /* Автоматические отступы слева и справа */
    border: 1px solid red;
}
```

Важно: Без заданной ширины элемент займет всю ширину родителя, и центрирование не сработает.

Центрирование с помощью позиционирования

Для точного позиционирования и наложения. Родителю задают position: relative, а элементу — position: absolute со смещением и трансформацией

```
<div class="parent">
    <div class="child">Я по центру родителя</div>
</div>
```

```
.parent {
    position: relative;
/* Точка отсчета */
    height: 200px;
    border: 2px solid
green;
}
```

```
.child {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);  /* Сдвиг на половину своих
размеров */
    background: lightblue;
    padding: 10px;
}
```

Центрирование с помощью позиционирования

Этот метод позволяет центрировать элемент поверх другого контента, даже если его размеры неизвестны.

1. `top: 50%; left: 50%` — помещает левый верхний угол элемента в центр родителя
2. `transform: translate(-50%, -50%)` — сдвигает элемент обратно на половину его собственной ширины и высоты

Плюсы

- Работает даже если размеры элемента неизвестны
- Элемент можно центрировать поверх другого контента
- Не влияет на остальные элементы (так как элемент вне потока)

Центрирование по вертикали для строчных элементов

Для строчных элементов внутри строки. Свойство line-height может помочь отцентрировать текст по вертикали (равен высоте блока)

```
.button {
    height: 50px;
    line-height: 50px;        /* Равно высоте блока */
    text-align: center;
    border: 1px solid black;
}
```

Подходит для: однострочного текста, кнопок

Центрирование с помощью табличных ячеек

Табличный способ (устаревший, но работает). `display: table-cell` с `vertical-align: middle`

```
.parent {
    display: table;
    width: 100%;
    height: 200px;
}

.child {
    display: table-cell;
    vertical-align: middle;   /* По вертикали */
    text-align: center;       /* По горизонтали */
}
```

Этот метод использует поведение таблиц, где `vertical-align` работает для ячеек.